Bleu Score

Tengo treinta y seis años

I have thirty six years

I am thirty six years old

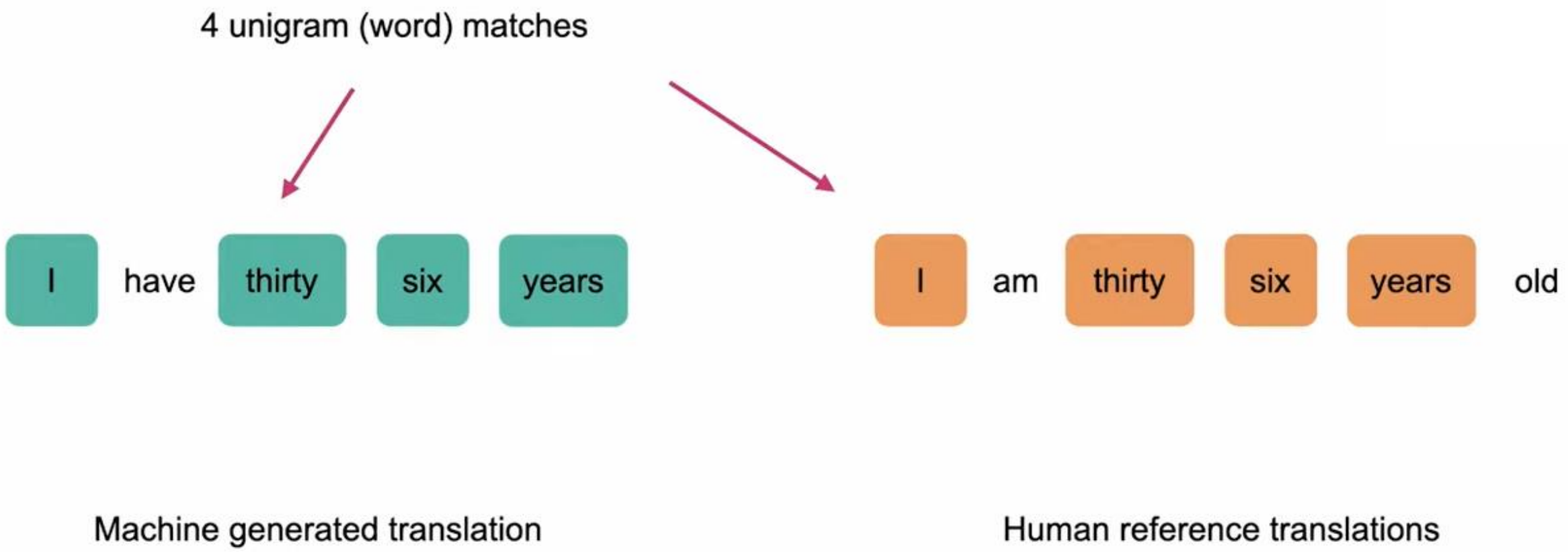
I am thirty six

Source text (Spanish)

Machine generated translation

Human reference translations

BLEU compares a generated translation to one or more reference translations



BLEU compares n-grams of the generation with n-grams of the references

$$\text{Unigram precision} = \frac{\text{Num word matches}}{\text{Num words in generation}} = \frac{4}{5}$$

I have thirty six years

Machine generated translation

I am thirty six years old

Human reference translations

BLEU is based on n-gram precision to measure translation quality

$$\text{Unigram precision} = \frac{\text{Num word matches}}{\text{Num words in generation}} = \frac{5}{5}$$

!?

six six six six six

I am thirty **six** years old

Machine generated translation

Human reference translations

But translation systems can overgenerate “reasonable” words …

$$\text{Modified unigram precision} = \frac{\text{Clip(Num word matches)}}{\text{Num words in generation}} = \frac{1}{5}$$

six six six six six

I am thirty six years old

Machine generated translation

Human reference translations

So BLEU uses a modified n-gram precision to clip the number of matches

$$\text{4-gram precision} = \frac{\text{Clip(Num 4-gram matches)}}{\text{Num 4-grams in generation}} = \frac{0}{5}$$

years six thirty have I

years six thirty have I

Machine generated translation

I am thirty six years old

I am thirty six years old

I am thirty six years old

Human reference translations

Here is an example of 4-gram precision (count chunks of 4 words)

```python
from datasets import load_metric

bleu = load_metric("bleu")
predictions = [["I", "have", "thirty", "six", "years"]]
references = [
    [["I", "am", "thirty", "six", "years", "old"], ["I", "am", "thirty", "six"]]
]
bleu.compute(predictions=predictions, references=references)
```

```
{'bleu': 0.0,
 'precisions': [0.8, 0.5, 0.3, 0.0],
 'brevity_penalty': 1.0,
 'length_ratio': 1.25,
 'translation_length': 5,
 'reference_length': 4}
```

unigram, bigram, trigram, 4-gram precision scores

You can compute BLEU scores in 🤗 Datasets with the load_metric() function

```python
from datasets import load_metric

bleu = load_metric("bleu")
predictions = [["I", "have", "thirty", "six", "years"]]
references = [
    [["I", "am", "thirty", "six", "years", "old"], ["I", "am", "thirty", "six"]]
]
bleu.compute(predictions=predictions, references=references)
```

```
{'bleu': 0.0,
 'precisions': [0.8, 0.5, 0.3, 0.0],
 'brevity_penalty': 1.0,
 'length_ratio': 1.25,
 'translation_length': 5,
 'reference_length': 4}
```

The "BLEU score" is the geometric mean of all four n-gram precisions

$$\text{BLEU-4} \sim \sqrt[4]{p_1 \cdot p_2 \cdot p_3 \cdot p_4}$$

You can compute BLEU scores in 🤗 Datasets with the load_metric() function

- ✅ Fast and simple to calculate
- ✅ Widely used

- ❌ Doesn't consider meaning
- ❌ Doesn't incorporate sentence structure
- ❌ Struggles with non-English languages
- ❌ Hard to compare scores with different tokenizers

See: *A Structured Review of the Validity of BLEU, Ehud Reiter (2018)*

Measuring translation quality is far from "solved"

Example: Reference 1: The cat is on the mat.

Reference 2: There is a cat on the mat.

MT output: The cat the cat on the mat.